the `rename_column` method:

```
class RenameEmailColumn < ActiveRecord::Migration
  def change
    rename_column :orders, :e_mail, :customer_email
  end
end
```

As `rename_column` is reversible, separate `up` and `down` methods aren't required in order to use it.

Note that the rename doesn't destroy any existing data associated with the column. Also be aware that renaming is not supported by all the adapters.

Changing Columns

`change_column` Use the `change_column` method to change the type of a column or to alter the options associated with a column. Use it the same way you'd use `add_column`, but specify the name of an existing column. Let's say that the order type column is currently an integer, but we need to change it to be a string. We want to keep the existing data, so an order type of `123` will become the string `"123"`. Later, we'll use noninteger values such as `"new"` and `"existing"`.

Changing from an integer column to a string is one line of code:

```
def up
  change_column :orders, :order_type, :string
end
```

However, the opposite transformation is problematic. We might be tempted to write the obvious `down` migration:

```
def down
 change_column :orders, :order_type, :integer
end
```

But if our application has taken to storing data like `"new"` in this column, the `down` method will lose it—`"new"` can't be converted to an integer. If that's acceptable, then the migration is acceptable as it stands. If, however, we want to